SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4316

COURSE OUTCOME COVERED

CO-1 : Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool : Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Code of Conduct:

I P.NIKITHA/192324226 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

P. Nikitha

Signature of the student:

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
   <input type="text" name="name">
   <input type="email" name="email">
   <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

1. Identify appropriate HTML5 semantic elements missing in the structure.

The given HTML form is missing appropriate HTML5 semantic elements that organize the webpage content.

- <header> – Defines the page heading or introductory content.
- <main> – Represents the main content of the webpage.
- <section> – Groups related symposium registration content.
- <form> – Contains the registration fields and submit button.
- <footer> – Defines footer information such as copyright details.
- <label> – Provides an accessible name for each input field.

2.Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Online Symposium Registration</title>
</head>
<body>

   <header>
      <h1>Online Symposium Portal</h1>
      <p>Register for the college symposium</p>
```

```
  </header>

  <main>
    <section>
      <h2>Student Registration</h2>

      <form action="/register" method="POST">
        <label for="name">Student Name:</label>
        <input type="text" id="name" name="name"
               autocomplete="name" required>
        <br><br>

        <label for="email">Email Address:</label>
        <input type="email" id="email" name="email"
               autocomplete="email" required>
        <br><br>

        <button type="submit">Submit</button>
      </form>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 College Symposium</p>
  </footer>

</body>
</html>
```

3. Interpret the HTTP request generated during form submission.

When the student submits the form, the browser sends an HTTP POST request to the server endpoint /register.

Example HTTP request:

```
POST /register HTTP/1.1
Host: college.edu
Content-Type: application/x-www-form-urlencoded

name=Ravi&email=ravi%40gmail.com
```

Explanation:

- POST: Sends the registration data to the server.
- /register: The server endpoint that processes registration.
- Host: Identifies the destination server.
- Content-Type: Specifies the format of the submitted data.
- Request body: Contains the student's name and email address.

The browser URL does not normally include the submitted form values when the POST method is used.

4. Analyze the HTTP response cycle from server to browser.

The HTTP communication follows these steps:

Step 1: Client (Browser)

Student fills and submits the form.

Step 2: HTTP POST Request

Registration data is sent to /register.

Step 3: Web Server

Validates the data and stores the registration if successful.

Step 4: HTTP Response

Server returns a status code and response content.

Step 5: Browser

Displays a success message or an error message.

Example HTTP response:

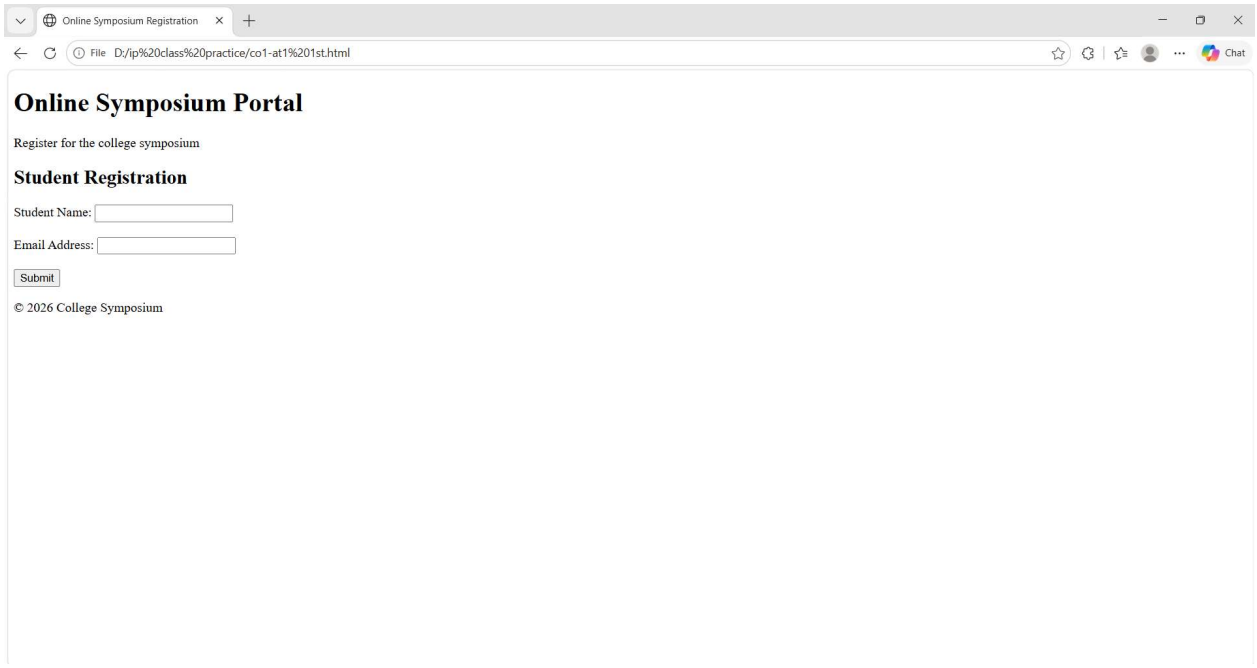
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

<h2>Registration Successful!</h2>

5. Suggest improvements for usability and accessibility.

1. Add labels: Associate each input with a <label> using matching for and id attributes.
2. Use required fields: Add required to mandatory inputs.

3. Validate email: Use type="email" for basic browser validation.
4. Improve keyboard access: Ensure all controls are keyboard-operable and show visible focus indicators.
5. Provide error messages: Explain invalid or missing information clearly.
6. Make it responsive: Use the viewport meta tag and suitable CSS for mobile devices.
7. Use HTTPS: Protect registration data during transmission.
8. Validate on the server: Never rely only on browser-side validation.
9. Protect against duplicate submissions: Prevent accidental repeated registrations.
10. Confirm successful registration: Display a clear confirmation message and accessible status updates.



2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
   <head>
      <title>Company</title>
   </head>
   <body>
      <div>
         <h1>Welcome
         <p>About us
      </div>
   </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.

2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

The given HTML code contains the following errors:

- The <h1> tag is missing its closing tag </h1>.
- The tag is missing its closing tag .
- The <html> tag is missing the language attribute lang="en".
- The document is missing the character encoding declaration <meta charset="UTF-8">.
- The viewport meta tag is missing for responsive design.
- The document does not use semantic HTML5 elements such as <header> and <main>.

2. Analyze how different browsers may interpret this structure.

Different browsers may handle malformed HTML differently because they use HTML parsing and error-recovery rules.

1. Google Chrome: Attempts to repair the malformed HTML and display the content.
2. Mozilla Firefox: Uses HTML error-recovery rules to construct a document structure.
3. Microsoft Edge: Attempts to correct missing tags and render the page.
4. Safari: May also recover from malformed markup, but layout and default styling can differ.

Result: Although browsers usually repair these errors, inconsistent document structures and default styles can cause differences in spacing, layout, and appearance.

3. Identify missing semantic elements.

The following HTML5 semantic elements can improve the website structure.

Element	Purpose
<header>	Defines introductory content or the page heading
<nav>	Contains navigation links, if required
<main>	Defines the primary page content

<section>	Groups related content
<footer>	Contains copyright or contact information

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Company</title>
</head>
<body>

   <header>
      <h1>Welcome</h1>
   </header>

   <main>
      <section>
         <h2>About Us</h2>
         <p>Learn more about our company and services.</p>
      </section>
   </main>

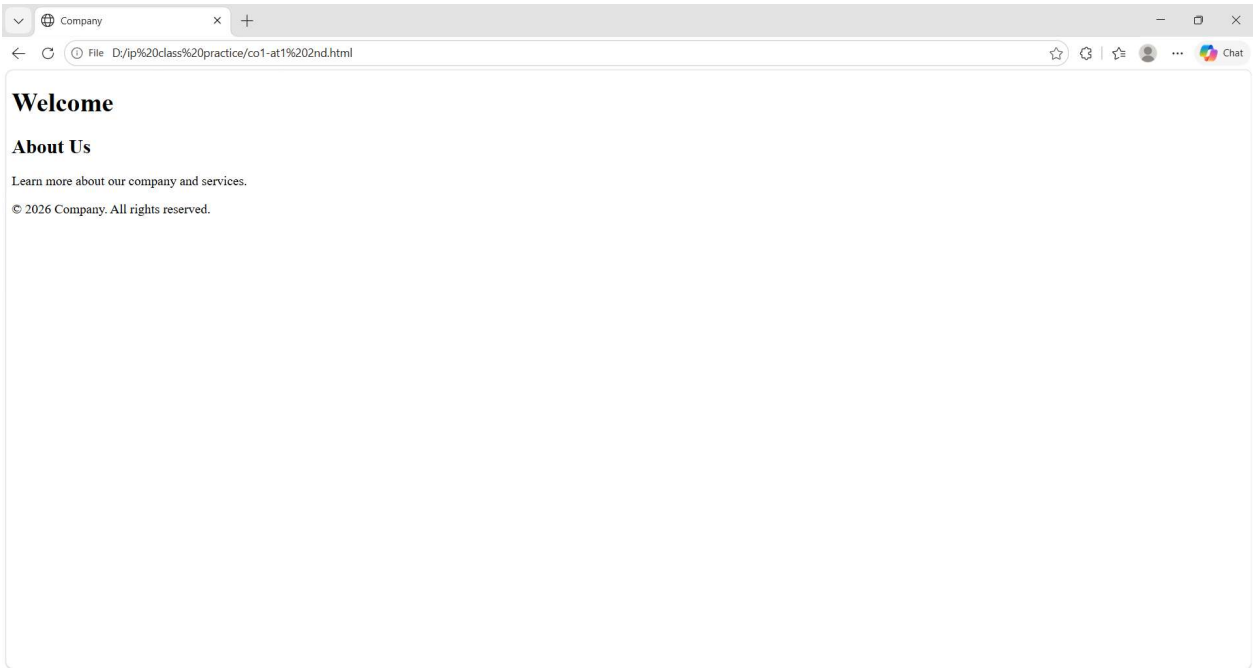
   <footer>
      <p>&copy; 2026 Company. All rights reserved.</p>
   </footer>

</body>
</html>
```

5. Suggest techniques to ensure cross-browser compatibility.

1. Use valid HTML: Close all tags and follow proper nesting rules.
2. Validate the code: Use the W3C HTML Validator at validator.w3.org.
3. Use CSS resets: Normalize default browser margins, padding, and other styles.
4. Use standard CSS: Prefer widely supported properties and avoid unnecessary browser-specific features.
5. Test multiple browsers: Check Chrome, Firefox, Edge, and Safari.
6. Use responsive design: Apply media queries and flexible layouts.
7. Specify character encoding: Use <meta charset="UTF-8">.
8. Include the viewport tag: Ensure proper display on mobile devices.
9. Use feature detection: Check browser support before using advanced features.

10. Maintain consistent styling: Use a common stylesheet and test different screen sizes.



3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
    <input type="text" name="username">
    <input type="password" name="password">
    <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

1. Identify issues in the form configuration.

The given HTML form has the following issues:

1. Incorrect button type: type="button" does not submit the form automatically.
2. Inappropriate HTTP method: GET places form data in the URL, which is unsuitable for passwords.
3. Missing required attributes: The username and password fields do not have required.
4. Missing labels: The input fields do not have labels for accessibility.

5. Security issue: Sending passwords through a GET request can expose them in browser history, URLs, and server logs.

2. CODE:

```
<button type="button">Login</button>
```

3. Interpret the role of GET method in this scenario.

The GET method retrieves information from a server. Form values are generally appended to the URL as query parameters.

Example:

GET /submit?username=Ravi&password=12345 HTTP/1.1
Host: example.com

4.CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Login Form</title>
</head>
<body>

   <h2>User Login</h2>

   <form action="/submit" method="POST">
      <label for="username">Username:</label>
      <input type="text" id="username"
           name="username" required>
      <br><br>

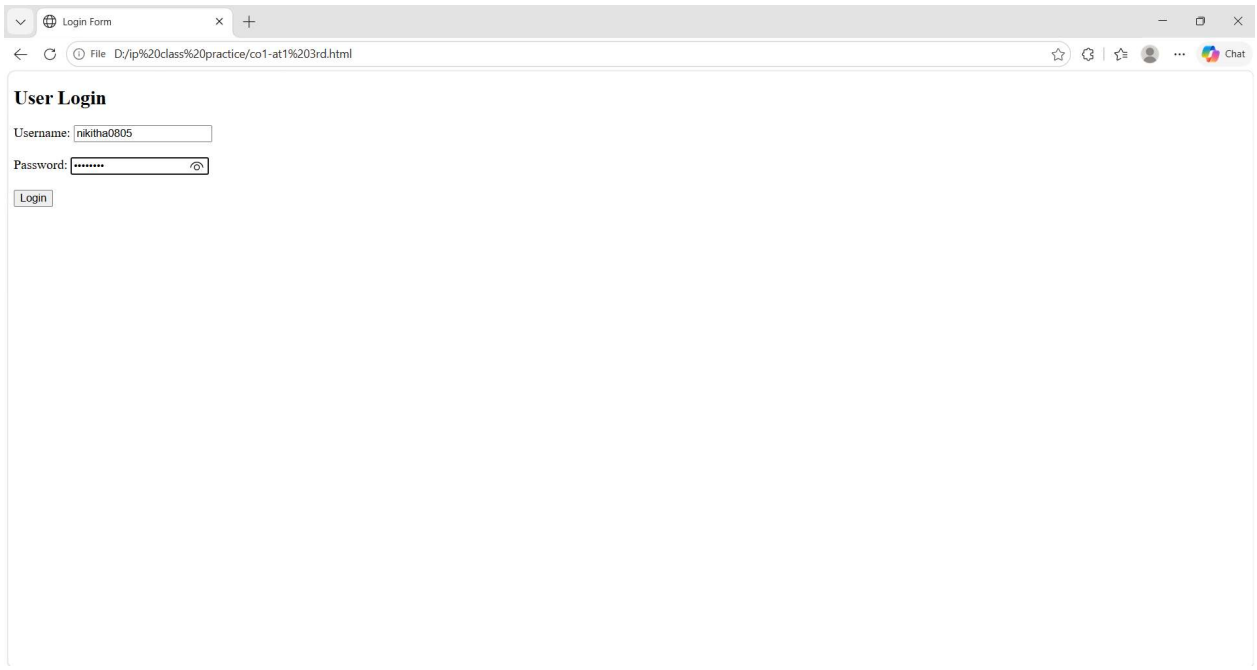
      <label for="password">Password:</label>
      <input type="password" id="password"
           name="password" required>
      <br><br>

      <button type="submit">Login</button>
   </form>
```

</body>
</html>

5. Suggest improvements for secure data transmission.

1. Use HTTPS: Encrypt data between the browser and server using TLS.
2. Use POST: Avoid placing credentials in the URL.
3. Validate on the server: Verify credentials on the server rather than relying only on HTML validation.
4. Hash passwords securely: Store passwords using a suitable password-hashing algorithm such as Argon2id or bcrypt.
5. Prevent brute-force attacks: Apply rate limiting and login-attempt controls.
6. Use secure sessions: Configure session cookies with Secure, HttpOnly, and appropriate SameSite attributes.
7. Avoid logging passwords: Never store plaintext passwords in application logs.
8. Display safe error messages: Avoid revealing whether a username or password alone was incorrect.



RUBRICS FOR CALCULATION:

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconception s	Poor understandin g; major misconceptio ns
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand